A Comparative Analysis of GEMM, FFT, and Winograd Convolution Algorithms in NVIDIA cuDNN

Prabin Karki, Samiksha Dhakal
Department of Electronics and Computer Engineering
Thapathali Campus, IOE, Tribhuvan University
Kathmandu, Nepal
karkeeprav67@gmail.com dsameeksha13@gmail.com

***Abstract*—Convolution is one of the most computationally intensive operations in convolutional neural networks. NVIDIA cuDNN provides multiple algorithms to accelerate its execution on GPU architectures. This work presents an empirical comparison of three convolution approaches: GEMM, FFT, and Winograd. The comparison covers execution time, GPU memory requirements, and numerical accuracy. Experiments were conducted on an NVIDIA Tesla T4 GPU using Google Colab with PyTorch 2.11.0+cu128, CUDA 12.8, and cuDNN 9.1.9. To isolate the effect of convolution parameters, all experiments used 64 input channels, 64 output channels, a spatial input size of 224×224, and FP32 precision. Batch sizes of 1, 8, and 32 and filter sizes of 1×1, 3×3, 5×5, 7×7, and 11×11 were evaluated. The algorithms were explicitly forced where supported by cuDNN. This allowed their execution time and workspace requirements to be compared independently. The results show that GEMM provides the lowest execution time for small filters and requires essentially no additional algorithm-specific workspace. FFT becomes more competitive for larger filters and batch sizes. Winograd provides substantial speed advantages for supported 3×3 and 5×5 convolutions but requires considerably more workspace. The memory analysis further demonstrates that workspace requirements can exceed the memory occupied by the convolution tensors by a large margin. These results highlight the trade-off between computational speed, workspace consumption, filter size, and batch size in GPU convolution algorithm selection.**

***Index Terms*—Convolutional Neural Networks, cuDNN, GEMM, FFT, Winograd, GPU Computing, Deep Learning, CUDA, PyTorch.**

I. Introduction

Convolutional Neural Networks (CNNs) remain the dominant architecture for computer vision tasks such as image classification, object detection, and segmentation. The convolution operation is the computational core of these networks [2]. As CNN models have grown deeper and process higher-resolution inputs, convolution now makes up most of the computational workload during both training and inference [1], [2].

Graphics Processing Units (GPUs) have sped up CNN computation considerably by exploiting the parallelism inherent in convolution. A naive direct implementation of convolution on a GPU is not efficient. It involves heavy memory access overhead and many repeated arithmetic operations. To address this, researchers have developed optimized convolution algorithms that reformulate the computation into forms better suited to modern GPU architectures. This cuts computational overhead while keeping the result numerically correct [1], [3].

NVIDIA's cuDNN library provides highly optimized GPU primitives for deep learning operations. These include convolution, pooling, normalization, and activation functions [1]. Rather than relying on a single implementation, cuDNN incorporates multiple convolution algorithms and dynamically selects the most efficient one based on parameters such as kernel size, input dimensions, batch size, stride, padding, and available GPU memory [1]. This adaptive approach allows cuDNN to perform well across a wide range of CNN architectures and hardware platforms.

Among the convolution algorithms implemented in cuDNN, three techniques are particularly significant: the General Matrix Multiplication (GEMM)-based approach, the Fast Fourier Transform (FFT)-based approach, and the Winograd minimal filtering algorithm [1], [2].

This paper presents a comparative study of these three algorithms. It covers their mathematical foundations, computational complexity, memory requirements, and practical performance. It also examines the conditions under which each algorithm performs best and discusses the trade-offs that shape algorithm selection in GPU-accelerated deep learning frameworks.

II. Literature Review

This section reviews the literature underlying the convolution algorithms examined in this paper. Each subsection addresses a specific computational strategy. It discusses the problem it was designed to address, the method proposed, the reported improvements, and the limitations identified in subsequent work.

A. cuDNN: The Standardization Gap

Before 2014, deep learning frameworks such as Caffe, Theano, and Torch each built and optimized their own GPU kernels separately. The same work had to be redone whenever new parallel architectures appeared. General scientific computing had already solved this problem through standardized libraries

such as the Basic Linear Algebra Subprograms (BLAS). No equivalent existed for deep learning. Chetlur et al. [1] addressed this gap by introducing cuDNN. It is a standardized library of GPU-accelerated primitives for deep learning that included convolution, pooling, and activation functions. The authors reported that adding cuDNN to Caffe improved performance by 36% on a standard model while also reducing memory use. They noted that convolution in the first release still performed worse than plain matrix multiplication. That version relied only on a GEMM-based approach. FFT-based and Winograd-based methods were only added in later releases, building on separate lines of research described below.

B. GEMM-based Convolution

The GEMM-based method, referred to as "Direct" in the original cuDNN paper by Chetlur et al. [1], addresses the poor data locality and inefficiency of naive nested-loop convolution on GPU hardware. This paper uses "GEMM" throughout for consistency with the terminology used in Sections IV and later. It reformulates convolution as a single dense matrix multiplication. Input patches are unrolled into columns of a matrix (a step known as im2col) and then multiplied against a matrix of flattened filter weights using optimized General Matrix Multiply (GEMM) routines. This approach produced a 36% speedup with lower memory use compared to earlier framework-specific implementations. Later comparative studies, including Zlateski et al. [4] and related work on matrix-multiplication-mapped convolution, consistently point to memory overhead as the main drawback of this method. This overhead is caused by the duplication of overlapping input values across multiple columns of the unrolled matrix.

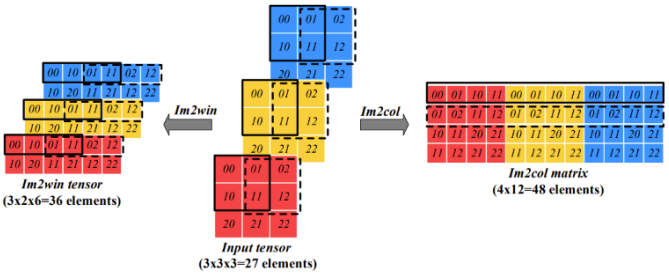

Fig. 1. The im2col and im2win data transformation examples with $C_i = H_i = W_i = 3$, $H_f = W_f = 2$, $s_h = s_w = 1$. The solid and dashed boxes indicate the different dot product windows of the input tensor. The im2win tensor has fewer elements than the im2col matrix. Reproduced from [10].

C. FFT based Convolution

The FFT-based method addresses a different problem in GEMM-based convolution. Its computational cost grows directly with filter size, which makes it inefficient for larger filters. Mathieu, Henaff, and LeCun [5] were the first to apply the convolution theorem systematically during CNN training. Their method transforms input feature maps and filters into the frequency domain, multiplies them element by element, and then transforms the result back using an inverse FFT. Their experiments showed a clear speedup over spatial-domain convolution. This advantage grew as filter size increased. Vasilache et al. [6] later extended this work with fbfft, an FFT implementation built specifically for GPU execution. It showed that a well-tuned FFT implementation could outperform GEMM-based convolution for certain layer configurations. Even with these gains, both studies point to a shared limitation: the transform overhead of FFT-based convolution hurts small filter sizes such as the 3×3 filters common in modern CNN architectures. The method also requires extra memory to store frequency-domain representations.

D. Winograd Minimal Filtering Convolution

The Winograd method addresses precisely the problem left open by the FFT approach: efficient computation for small, fixed-size filters. Its theoretical basis predates deep learning entirely. Winograd [7] developed minimal-multiplication algorithms for short linear filtering operations. These algorithms trade multiplications for additions through algebraic identities with no reference to neural networks. Lavin and Gray [2] were the first to adapt this theory to CNN convolution layers. They proposed practical algorithms designed for small filter sizes and reported a $2.25\times$ reduction in required multiplications for 3×3 filters compared to direct convolution. This is a significant result given that most modern CNN architectures are built mainly from filters of this size. Both Lavin and Gray [2] and later work by Barabasz et al. [8] identify a numerical stability issue: the transform matrices behind Winograd convolution become increasingly ill-conditioned as tile size grows. This amplifies floating-point error. Barabasz et al. [8] proposed corrections that improved numerical accuracy while largely preserving the speed advantage of the Winograd method. They note that larger tile sizes remain fundamentally more difficult to stabilize. Meng and Brothers [9] approached the same tradeoff differently by proposing an integer-arithmetic formulation of Winograd convolution to make it better suited for quantized, low-precision inference on hardware accelerators.

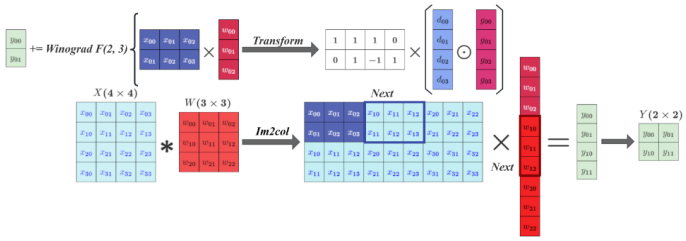

Fig. 2. Im2col-Winograd convolution workflow. Reproduced from [11].

E. Synthesis and Research Gap

The papers discussed so far each focus on a single algorithm. Relatively little work compares these methods directly on shared hardware. Zlateski, Jia, Li, and Durand [4] address this gap by implementing efficient FFT-based and Winograd-based convolution and evaluating both across three distinct processor architectures (Intel Skylake, ARM Cortex A57, and Fujitsu

A64FX). Their results indicate that Winograd convolution, despite its numerical precision tradeoff, remains competitive with GEMM-based convolution and performs particularly well for image segmentation tasks. This study is limited to CPU architectures. In particular, no existing work has empirically compared Direct, FFT, and Winograd convolution within cuDNN itself across varying filter size, batch size, and channel depth on GPU hardware. This is the gap the present paper addresses.

Taken together, the literature indicates that no single convolution algorithm is universally optimal. The Direct method offers generality at the cost of memory overhead. The FFT method offers favorable scaling for large filters at the cost of overhead on small ones. Winograd achieves lower arithmetic cost than Direct or FFT convolution specifically for small, fixed filter sizes such as 3×3 on hardware and precision settings where its transform overhead and numerical error remain acceptable. This advantage does not generalize to larger filters or higher-tile configurations [2], [8]. This progression is reflected in cuDNN's own evolution: the original release implemented only GEMM-based convolution [1], while Winograd-based and FFT-based methods were incorporated in later versions, building on the separate lines of research discussed above [2], [5], [6]. Subsequent releases also introduced heuristic and empirical auto-tuning mechanisms to select the most suitable algorithm on a per-layer basis depending on filter size, batch size, and available memory. The present paper builds on this literature by providing a direct GPU-specific comparative analysis of all three methods within the cuDNN framework. This addresses the CPU-only limitation of prior comparative work identified above.

III. Methodology

A. Theoretical Framework and Justification

The comparison presented in this paper rests on the premise that the GEMM-based, FFT-based, and Winograd-based convolution algorithms are mathematically equivalent in exact arithmetic. For a given input tensor and filter, all three methods are defined to compute the same convolution result, given for output position (q, k, p, g) as

$$O[q,k,p,g] = \sum_{c} \sum_{r} \sum_{s} F[k,c,r,s] \cdot D[q,c,\, p \cdot u + r,\, g \cdot v + s] \tag{1}$$

where C, R, S denote the number of input channels and filter height/width. u and v denote the stride [1]. In finite-precision floating-point computation this equivalence holds only approximately. The three algorithms reorder the underlying additions and multiplications differently: Winograd's algebraic transform and FFT's frequency-domain multiplication both change the order in which values are summed compared with direct accumulation. This can introduce small numerical differences even for identical inputs. This paper does not assume exact numerical equivalence in practice. Instead, Section IV-C reports the numerical deviation observed between algorithms under the tested conditions. This equivalence claim is intended only as the theoretical basis for treating the three algorithms as comparable on efficiency grounds.

To structure the comparison consistently, this paper adopts three evaluation dimensions drawn from the literature reviewed in Section II. **Arithmetic complexity** covers, for example, $O(n^2k^2)$ for Direct convolution and $O(n^2 \log n)$ for FFT-based convolution, or a fixed multiplication-reduction factor for Winograd on small, fixed-size filters [2]. **Memory complexity** covers overhead introduced by the im2col matrix, frequency-domain buffers, or transformed tiles for each method. **Numerical stability** covers floating-point error relative to direct computation, reported as particularly significant for the Winograd method at larger tile sizes [8]. This framework mirrors the tradeoffs that motivated cuDNN's design of implementing all three algorithms and selecting between them on a per-layer basis [1]. It is further grounded in the observation that convolution's high degree of data parallelism is what makes GPU acceleration effective for this operation in the first place [3].

B. Research Design

This study follows a mixed comparative-analytical and empirical research design. The first component synthesizes and critically compares theoretical complexity results and benchmark findings already reported in the literature reviewed in Section II. This includes Chetlur et al. [1], Lavin and Gray [2], Mathieu et al. [5], Vasilache et al. [6], and Zlateski et al. [4]. The second component consists of original empirical benchmarking of the three convolution algorithms on GPU hardware. It was conducted to determine whether the performance trends established in this literature hold on currently available GPU hardware. This literature is largely reported on earlier GPU generations or, in the case of Zlateski et al. [4], on CPU architectures. This directly addresses the GPU-specific gap identified in Section II-E.

The empirical comparison is organized around the same parameters that govern algorithm selection within cuDNN itself [1]:

1) **Filter size**: evaluated at 1×1, 3×3, 5×5, 7×7, and 11×11. Filter size is the strongest determinant of relative algorithm performance.
2) **Batch size**: evaluated at 1, 8, and 32 for each configuration. Batch size affects both the relative cost of memory movement versus arithmetic computation and the total peak memory consumed. This makes it a necessary variable for the memory-complexity comparison conducted in Section IV.
3) **Input channel count and spatial resolution**: varied jointly with filter size across configurations (ranging from 3 to 128 input channels and from 28×28 to 224×224 spatial resolution) rather than held fixed. This was done in order to examine whether filter size alone determines cuDNN's algorithm selection or whether channel depth and resolution are also contributing factors. This design choice was informed by a preliminary observation: two configurations sharing an identical 3×3 filter size but

differing only in input channel count were assigned different cuDNN algorithms. This indicates that channel count is not a controllable nuisance variable in this context but a variable of direct empirical interest.

4) **Layer configuration realism**: each combination of filter size, channel count, and spatial resolution was selected to correspond to an identifiable layer within a well-known CNN architecture (AlexNet, ResNet-50, VGG-16, and an Inception-style block). This ensures the resulting comparison reflects conditions encountered in practical deployment rather than arbitrary parameter combinations.

The dependent variables measured are GPU execution latency, peak memory consumption, and the specific cuDNN algorithm selected for each configuration.

C. Data Collection Procedure

Two parallel data collection tracks were used, corresponding to the two components of the research design.

1) Literature-based collection: Sources were identified through Google Scholar, arXiv, and the ACM/IEEE digital libraries using search terms including "cuDNN convolution algorithm," "Winograd convolution neural network," "FFT convolution GPU," and "im2col GEMM convolution." Sources were prioritized based on direct relevance to GPU-based convolution algorithms, venue and citation count, and whether reproducible performance figures were reported. This is consistent with the synthesis in Section II.

2) Empirical benchmarking: Experiments were conducted on an NVIDIA Tesla T4 GPU through Google Colab using PyTorch 2.11.0+cu128, CUDA 12.8, and cuDNN 9.1.9. The convolution configuration was kept constant at 64 input channels, 64 output channels, and 224×224 input dimensions with FP32 precision. Batch sizes of 1, 8, and 32 and filter sizes of 1×1, 3×3, 5×5, 7×7, and 11×11 were evaluated. GEMM, FFT, and Winograd algorithms were explicitly forced for each supported configuration. Execution time, allocated GPU memory, peak memory, and algorithm-specific workspace requirements were recorded. Unsupported Winograd configurations were reported as "–". Stride and padding were not held constant or explicitly logged across the tested filter sizes in this experiment. This is a limitation when interpreting how output spatial dimensions, and by extension the memory results in Table III, vary with filter size. Reporting these values explicitly for each configuration is left for a future revision. The measurements were then compared across different filter and batch sizes.

D. Data Processing and Analysis Tools

PyTorch was used to construct and execute the convolution operations and to access the cuDNN convolution implementations. CUDA event timers were used to measure GPU execution time. GPU synchronization was performed to ensure accurate timing. GPU memory statistics were obtained using PyTorch CUDA memory-management functions. The workspace requirement associated with each forced cuDNN algorithm was recorded during algorithm execution.

Unlike an autotuning-based experiment, the algorithm was not inferred from execution time or automatically selected by cuDNN. Instead, GEMM, FFT, and Winograd were explicitly specified and benchmarked independently. This allowed the execution-time and memory behavior of each algorithm to be directly compared under identical input, channel, precision, and spatial configurations.

Python was used to execute the benchmark, collect the measurements, and organize the results. The collected data were subsequently analyzed to compare the effects of filter size and batch size on execution time, allocated memory, and workspace requirements.

IV. Result

The three convolution algorithms available in cuDNN (GEMM, FFT, and Winograd) were systematically evaluated on an NVIDIA Tesla T4 GPU using PyTorch 2.11.0+cu128, CUDA 12.8, and cuDNN 9.1.9. The experiments used a fixed convolution configuration with 64 input channels, 64 output channels, and an input size of 224×224. Filter sizes of 1×1, 3×3, 5×5, 7×7, and 11×11 were evaluated at batch sizes of 1, 8, and 32. Rather than relying on automatic algorithm selection, each supported algorithm was explicitly forced and benchmarked independently. Execution time, allocated GPU memory, peak memory, and algorithm-specific workspace requirements were recorded for each configuration. cuDNN's Winograd implementation only supports the 3×3 and 5×5 filter sizes tested here. The 1×1, 7×7, and 11×11 configurations therefore have no corresponding Winograd kernel and are reported as "–" in Tables I, III, and IV. The results were then compared to analyze the performance and memory characteristics of GEMM, FFT, and Winograd across different filter and batch sizes.

A. Time Complexity Analysis

The execution time of GEMM, FFT, and Winograd convolution algorithms was evaluated for filter sizes of 1×1, 3×3, 5×5, 7×7, and 11×11 with batch sizes of 1, 8, and 32. All other convolution parameters, including the input spatial dimensions, input channels, output channels, precision, stride, and padding, were kept constant. The experiments were performed on an NVIDIA Tesla T4 GPU using FP32 precision. The measured execution times are presented in Table I.

The results in Table I demonstrate that the relative performance of the convolution algorithms depends strongly on both filter size and batch size. GEMM provides the lowest execution time for the 1×1 filter across all tested batch sizes. For example, at batch size 32, GEMM requires only 12.5798 ms compared with 68.0573 ms for FFT.

As the filter size increases, the execution time of GEMM increases substantially. At batch size 32, its execution time increases from 12.5798 ms for 1×1 to 756.3433 ms for 11×11. This corresponds to an increase of

$$\frac{756.3433}{12.5798} \approx 60.12. \tag{2}$$

TABLE I
EXECUTION TIME OF CUDNN CONVOLUTION ALGORITHMS

Batch	Filter	GEMM	FFT	Winograd
1	1×1	0.7659	33.0823	–
	3×3	1.9142	33.5676	8.8516
	5×5	5.3746	33.6822	14.6682
	7×7	10.0742	33.8945	–
	11×11	24.6233	34.0812	–
8	1×1	3.0582	30.4661	–
	3×3	17.3617	31.5952	10.6921
	5×5	43.8457	31.7765	18.5070
	7×7	81.0824	31.3904	–
	11×11	192.1354	30.8517	–
32	1×1	12.5798	68.0573	–
	3×3	65.6650	70.8165	23.6010
	5×5	164.6635	71.5564	40.2716
	7×7	319.4939	73.6957	–
	11×11	756.3433	71.8721	–

Thus, the measured GEMM execution time for the 11×11 filter is approximately 60.1 times that of the 1×1 filter at batch size 32.

Winograd demonstrates a significant advantage for the supported 3×3 and 5×5 filters at larger batch sizes. For the 3×3 filter with batch size 32, the speedup of Winograd over GEMM is

$$S_{\mathrm{W}} = \frac{65.6650}{23.6010} \approx 2.78 \times . \tag{3}$$

For the 5×5 filter, the corresponding speedup is

$$S_{\mathrm{W}} = \frac{164.6635}{40.2716} \approx 4.09 \times . \tag{4}$$

Therefore, Winograd reduces the execution time by approximately 64.0% and 75.5% compared with GEMM for the 3×3 and 5×5 filters respectively at batch size 32.

For larger filters, FFT becomes increasingly advantageous. At batch size 32, the 7×7 filter requires 319.4939 ms using GEMM but only 73.6957 ms using FFT. The corresponding speedup is

$$S_{\mathrm{FFT}} = \frac{319.4939}{73.6957} \approx 4.34 \times . \tag{5}$$

For the 11×11 filter, FFT achieves

$$S_{\mathrm{FFT}} = \frac{756.3433}{71.8721} \approx 10.52\times \tag{6}$$

speedup over GEMM. This indicates that the advantage of FFT becomes more pronounced as the filter size increases.

It is worth noting that FFT execution time does not increase monotonically with batch size unlike GEMM and Winograd. At the 1×1 filter, FFT takes approximately 33.08 ms at batch size 1, drops to 30.47 ms at batch size 8, then rises to 68.06 ms at batch size 32. A plausible explanation is that cuFFT plan creation and allocation overhead do not scale linearly with batch size. Batch sizes 1 and 8 may benefit from more favorable plan caching or memory reuse than batch size 32. Isolating the exact cause would require profiling cuFFT plan behavior directly. This is left for future work.

The overall fastest algorithm observed for each configuration is summarized in Table II.

TABLE II
FASTEST ALGORITHM FOR EACH CONFIGURATION

Batch	1×1	3×3	5×5	7×7	11×11
1	GEMM	GEMM	GEMM	GEMM	GEMM
8	GEMM	Winograd	Winograd	FFT	FFT
32	GEMM	Winograd	Winograd	FFT	FFT

As shown in Table II, GEMM remains the fastest method for 1×1 convolution for all tested batch sizes. At batch sizes 8 and 32, Winograd becomes the fastest method for 3×3 and 5×5 filters. FFT becomes the fastest for 7×7 and 11×11 filters. This demonstrates that no single convolution algorithm provides optimal performance for all configurations.

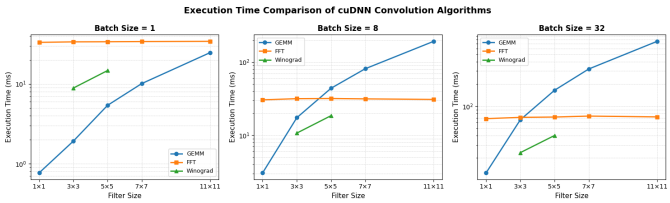

Fig. 3. Execution time of GEMM, FFT, and Winograd for different filter sizes and batch sizes.

Figure 3 further illustrates the difference in execution time among the algorithms. GEMM shows a steep increase in execution time with increasing filter size, particularly at larger batch sizes. FFT exhibits comparatively stable execution time for larger filters. Winograd provides lower execution time for the supported 3×3 and 5×5 configurations.

Overall, the experimental results show that GEMM is most effective for 1×1 convolutions, Winograd is advantageous for small spatial filters at larger batch sizes, and FFT becomes increasingly effective for large filters. These observations demonstrate the dependence of convolution performance on filter size and batch size and provide experimental support for workload-dependent convolution algorithm selection in cuDNN.

B. Memory Consumption Analysis

The memory requirements of GEMM, FFT, and Winograd were evaluated using the same experimental configuration as the time analysis. The input spatial dimensions were fixed at 224×224, with 64 input channels, 64 output channels, and FP32 precision. Only the batch size and filter size were varied.

Two main memory characteristics were analyzed: allocated GPU memory and algorithm-specific workspace memory. Allocated memory represents the memory occupied by the input, weights, output, and other persistent tensors. Workspace memory represents temporary GPU memory required by the selected convolution algorithm during execution.

Table III shows that the allocated memory is nearly identical for GEMM, FFT, and Winograd for the same configuration. This is because allocated memory is primarily determined

TABLE III
ALLOCATED GPU MEMORY FOR DIFFERENT BATCH AND FILTER SIZES

Batch	Filter	GEMM (MB)	FFT (MB)	Winograd (MB)
1	1×1	36.91	36.91	–
1	3×3	37.03	37.03	37.03
1	5×5	37.28	37.28	37.28
1	7×7	37.66	37.66	–
1	11×11	38.78	38.78	–
8	1×1	122.66	122.66	–
8	3×3	122.78	122.78	122.78
8	5×5	123.03	123.03	123.03
8	7×7	123.41	123.41	–
8	11×11	124.53	124.53	–
32	1×1	416.66	416.66	–
32	3×3	416.78	416.78	416.78
32	5×5	417.03	417.03	417.03
32	7×7	417.41	417.41	–
32	11×11	418.53	418.53	–

TABLE IV
WORKSPACE MEMORY REQUIREMENTS OF CUDNN CONVOLUTION ALGORITHMS (MB)

B	Filter	GEMM	FFT	Winograd
1	1×1	0.00	2096.27	–
1	3×3	0.00	2096.39	882.56
1	5×5	0.00	2096.64	827.84
1	7×7	0.00	2097.02	–
1	11×11	0.00	2098.14	–
8	1×1	0.00	2322.02	–
8	3×3	0.00	2322.14	882.56
8	5×5	0.00	2322.39	827.84
8	7×7	0.00	2322.77	–
8	11×11	0.00	2323.89	–
32	1×1	0.00	3096.02	–
32	3×3	0.00	3096.14	1764.56
32	5×5	0.00	3096.39	1653.03
32	7×7	0.00	3096.77	–
32	11×11	0.00	3097.89	–

by the input, weight, and output tensors rather than by the convolution algorithm itself.

For example, for a batch size of 32 and a 5×5 filter, all three supported algorithms require approximately 417.03 MB of allocated memory. The dominant factor is therefore the batch size.

For the 1×1 convolution, the allocated memory increases from 36.91 MB at batch size 1 to 416.66 MB at batch size 32. This corresponds to an approximately $11.29\times$ increase:

$$\frac{416.66}{36.91} \approx 11.29. \tag{7}$$

This demonstrates that persistent tensor memory increases substantially with batch size, while the effect of filter size on allocated memory is comparatively small.

The workspace requirements show a substantially different behavior among the convolution algorithms, as shown in Table IV. GEMM requires no additional workspace for any of the tested configurations. FFT requires considerably more temporary memory, ranging from approximately 2.10 GB at batch size 1 to approximately 3.10 GB at batch size 32.

For example, at batch size 32 and a 3×3 filter, FFT requires 3096.14 MB of workspace, whereas Winograd requires only 1764.56 MB. The relative difference can be expressed as

$$\frac{3096.14}{1764.56} \approx 1.75. \tag{8}$$

Thus, FFT requires approximately $1.75\times$ more workspace than Winograd for this configuration.

Similarly, for the 5×5 filter at batch size 32,

$$\frac{3096.39}{1653.03} \approx 1.87. \tag{9}$$

Hence, FFT requires approximately $1.87\times$ more workspace than Winograd for the 5×5 configuration.

The FFT workspace also increases with batch size. For the 1×1 filter, it increases from 2096.27 MB at batch size 1 to 3096.02 MB at batch size 32. This represents an increase of approximately 47.7%:

$$\frac{3096.02 - 2096.27}{2096.27} \times 100 \approx 47.7\%. \tag{10}$$

TABLE V
COMPARISON OF FFT AND WINOGRAD WORKSPACE

Batch	Filter	FFT (MB)	Winograd (MB)	FFT/Winograd
1	3×3	2096.39	882.56	$2.38\times$
1	5×5	2096.64	827.84	$2.53\times$
8	3×3	2322.14	882.56	$2.63\times$
8	5×5	2322.39	827.84	$2.81\times$
32	3×3	3096.14	1764.56	$1.75\times$
32	5×5	3096.39	1653.03	$1.87\times$

Hence, the memory analysis demonstrates that allocated memory and workspace memory have different dependencies. Allocated memory is primarily determined by the tensor dimensions and therefore increases substantially with batch size. Workspace memory is strongly dependent on the selected convolution algorithm.

GEMM exhibited the lowest memory overhead, requiring 0 MB of additional workspace in all tested configurations. FFT exhibited the highest workspace requirement, reaching approximately 3.10 GB at batch size 32. Winograd required an intermediate amount of workspace for the configurations in which it was supported.

These observations demonstrate a clear time–memory trade-off among the convolution algorithms. GEMM provides the lowest workspace requirement but becomes slower as the filter size increases. FFT requires substantially more temporary memory but provides favorable execution time for large filters. Winograd provides a compromise between execution time and workspace requirements for supported filter sizes.

Therefore, the results indicate that the fastest convolution algorithm is not necessarily the most memory-efficient algorithm. The choice of convolution algorithm involves a trade-off between execution time and GPU memory consumption.

C. Numerical Stability Analysis

Numerical stability was evaluated by comparing the convolution outputs with a reference result using the maximum absolute error. Lower error values indicate better numerical

agreement with the reference computation. The results are summarized in Table VI.

TABLE VI
MAXIMUM ABSOLUTE ERROR FOR DIFFERENT CONVOLUTION CONFIGURATIONS

Batch	Filter	GEMM	FFT	Winograd
1	1×1	0.000013	0.000013	0.000013
1	3×3	0.000045	0.000045	0.000045
1	5×5	0.000371	0.000371	0.000371
1	7×7	0.000734	0.000734	0.000734
1	11×11	0.001783	0.001783	0.001783
8	1×1	0.000018	0.000018	0.000018
8	3×3	0.000051	0.000051	0.000051
8	5×5	0.000429	0.000429	0.000429
8	7×7	0.000897	0.000897	0.000897
8	11×11	0.002449	0.002449	0.002449
32	1×1	0.000026	0.000026	0.000026
32	3×3	0.001115	0.001115	0.001115
32	5×5	1.791983	1.791983	1.791983
32	7×7	0.000102	0.000102	0.000102
32	11×11	0.000153	0.000153	0.000153

As shown in Table VI, the numerical error is generally very small for most configurations. For batch size 1, the maximum absolute error increases from approximately 1.35×10^{-5} for the 1×1 filter to 1.78×10^{-3} for the 11×11 filter. This indicates that larger filters can introduce greater floating-point discrepancies because of the increased number of arithmetic operations.

The effect of batch size is less consistent. For the 1×1 filter, the maximum error increases from 1.35×10^{-5} at batch size 1 to 2.58×10^{-5} at batch size 32. The batch-32 5×5 configuration shows an anomalously large error of approximately 1.79. This is roughly four orders of magnitude larger than any other entry in Table VI. Given the abrupt discontinuity relative to every neighboring configuration, this value is treated as a measurement anomaly rather than a reliable numerical-stability observation and is excluded from the general trend discussed above. Identifying the underlying cause (e.g., a numerical overflow, a buffer artifact, or a shape-handling issue specific to that configuration) is left for future work. Numerical accuracy was evaluated using FP32 convolution against a higher-precision/reference computation.

GEMM, FFT, and Winograd exhibit identical error values in the current measurements. Therefore, no algorithm-specific difference in numerical stability can be established from these results. Overall, the convolution computations demonstrate good numerical agreement for most configurations. Larger filters and the identified anomalous configuration exhibit comparatively higher numerical errors.

V. CONCLUSION

This work presented an empirical evaluation of GEMM, FFT, and Winograd convolution algorithms available in NVIDIA cuDNN. The experiments were performed on an NVIDIA Tesla T4 GPU using PyTorch 2.11.0+cu128, CUDA 12.8, and cuDNN 9.1.9. A fixed convolution configuration of 64 input channels, 64 output channels, and 224×224 spatial dimensions was used, while batch size and filter size were varied independently.

The execution-time results demonstrate that no single algorithm is optimal for all convolution configurations. GEMM provides the lowest latency for small filters and remains effective as the filter size increases at small batch sizes. Its execution time grows rapidly for larger filters and batch sizes. FFT exhibits relatively stable execution time across filter sizes and becomes considerably more competitive for larger filters and larger batch sizes. Winograd achieves the best performance among the tested algorithms for the supported 3×3 and 5×5 configurations, particularly at larger batch sizes, but is not supported by the tested cuDNN configuration for all filter sizes.

The memory analysis reveals a significant difference in workspace requirements between the algorithms. GEMM requires essentially zero additional workspace in the tested configurations, whereas FFT requires approximately 2.1–3.1 GB of workspace depending on batch size. Winograd requires less workspace than FFT but still consumes approximately 0.83–1.76 GB for the supported configurations. Thus, an algorithm with lower execution time may impose substantially higher memory requirements.

The numerical-stability experiment generally produced small errors relative to the reference computation for most configurations. The current implementation produced identical numerical errors for the three reported algorithms. Therefore, these results cannot be used to establish a difference in numerical stability between GEMM, FFT, and Winograd until algorithm-specific execution is independently verified during the numerical experiment. An isolated large error was also observed for the batch-32 5×5 configuration and should be investigated separately.

Overall, the experiments demonstrate that convolution algorithm selection involves a trade-off among execution time, filter size, batch size, and GPU memory availability. The results also show why algorithm selection cannot be based solely on filter size: the optimal method depends on the complete convolution configuration and the available GPU resources.

REFERENCES

[1] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cuDNN: Efficient primitives for deep learning," *arXiv preprint arXiv:1410.0759*, 2014.

[2] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 4013–4021.

[3] V. Volkov and J. Demmel, "Benchmarking GPUs to tune dense linear algebra," in *Proceedings of the ACM/IEEE Conference on Supercomputing (SC)*, Austin, TX, USA, 2008.

[4] A. Zlateski, Z. Jia, K. Li, and F. Durand, "The anatomy of efficient FFT and Winograd convolutions on modern CPUs," in *Proc. ACM Int. Conf. Supercomputing (ICS)*, 2019.

[5] M. Mathieu, M. Henaff, and Y. LeCun, "Fast training of convolutional networks through FFTs," *arXiv:1312.5851*, 2013.

[6] N. Vasilache, J. Johnson, M. Mathieu, S. Chintala, S. Piantino, and Y. LeCun, "Fast convolutional nets with fbfft: A GPU performance evaluation," *arXiv:1412.7580*, 2014.

[7] S. Winograd, *Arithmetic Complexity of Computations*, vol. 33. SIAM, 1980.

[8] B. Barabasz, A. Anderson, K. M. Soodhalter, and D. Gregg, "Improving the accuracy of Winograd convolution for convolutional neural networks," *ACM Trans. Math. Software*, 2020.
[9] L. Meng and J. Brothers, "Efficient Winograd convolution via integer arithmetic," *arXiv:1901.01965*, 2019.
[10] S. Lu, J. Chu, L. Guo, and X. T. Liu, "Im2win: An Efficient Convolution Paradigm on GPU," *arXiv preprint arXiv:2306.14316*, 2023.
[11] Z. Zhang, P. Zhang, Z. Xu, B. Yan, and Q. Wang, "Im2col-Winograd: An Efficient and Flexible Fused-Winograd Convolution for NHWC Format on GPUs," *arXiv preprint arXiv:2403.16404*, 2024.